Procedural Chinese Dragon Modeling

A Detailed Technical Breakdown

Author: Haomin GAO, Junxian GUAN, Mingyue ZHAO, Ruining DONG

Final Model Showcase

Full View

This is the complete dragon model.

Head Close-up

This detail shows the horns, eyes, and the beard.

Project Overview

Project Goal

To create a 3D model of a Chinese Dragon using procedural programming with OpenSCAD.

Methodology

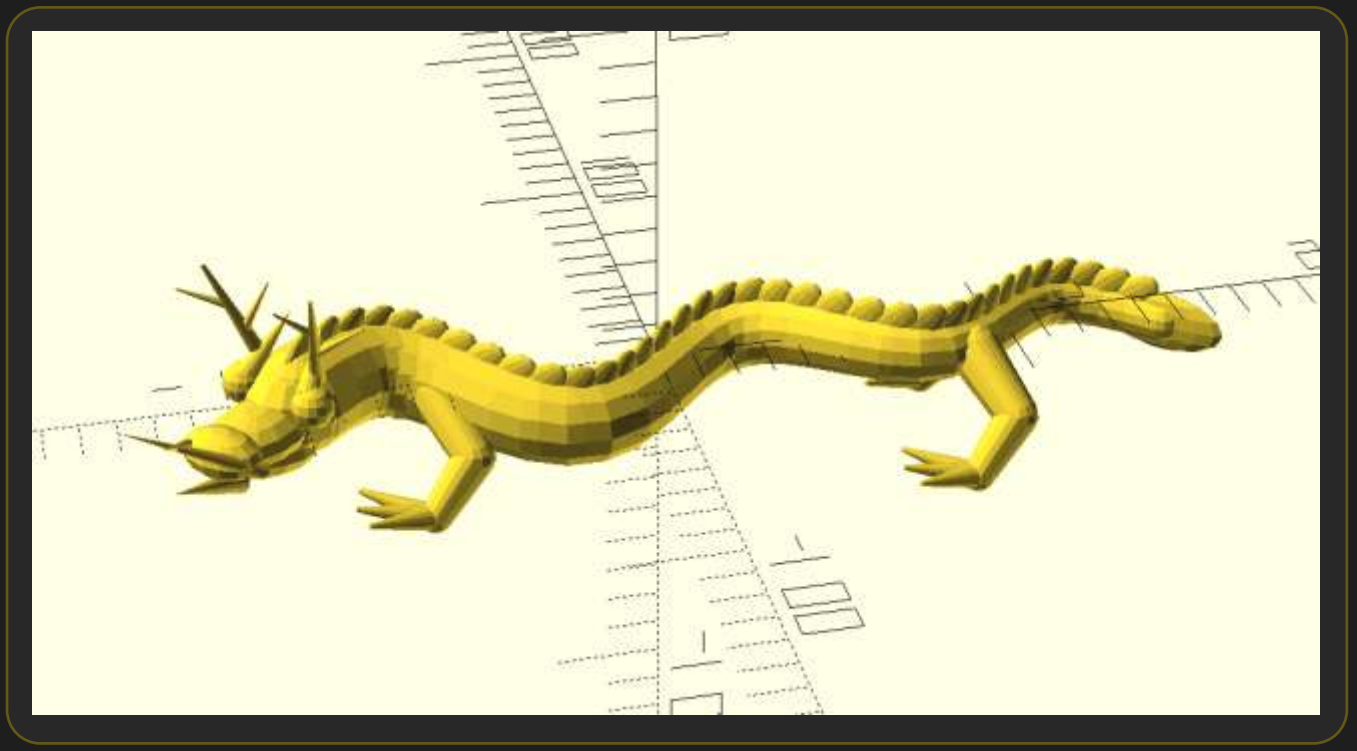
Break the dragon into modular parts (scales, beard, claws), model each separately, then assemble.

Key Tools

Utilizing OpenSCAD for geometric modeling and function calls to generate the 3D structure.

SUMMARY: This project demonstrates the power of procedural programming in complex 3D modeling, showcasing how modular design can simplify the creation of intricate objects like a Chinese Dragon.

Initial Model by Eric Guan



BACKGROUND: I asked Doubao AI to write a prompt for Gemini 3 Pro to generate a Chinese dragon model. And this is the model generated by Gemini 3 Pro.

Scale Modeling by Eric Guan

Method

The scales were made by first intersecting two spheres to form a base shape, then subtracting two perpendicular cylinders to carve out the final form.

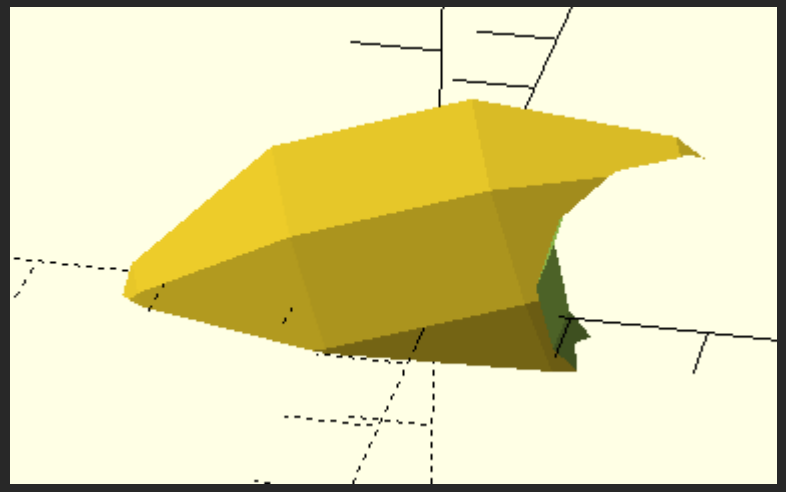
Key Functions

- **intersection():** Find the overlap of two spheres
- **difference():** Cut the cylinders out from the base

Parameters

- **radius:** Size of the spheres
- **offset_x/z:** Position of the spheres
- **cut_radius:** Size of cutting cylinders
- **angle:** Rotation of the final shape

Scale Modeling by Eric Guan



Parametric Design Workflow • 3D Modeling Process

Beard Modeling by Eric Guan

Core Methodology

Created using a **for-loop** to iterate along a curve. Two small spheres are placed in each step, then connected with the **hull()** function to form a smooth, flowing surface.

Key Functions

- **for-loop:** Repeats the sphere placement process efficiently.
- **hull():** Connects geometry smoothly to eliminate sharp edges.

Curve Formula

The beard path follows a sine curve:

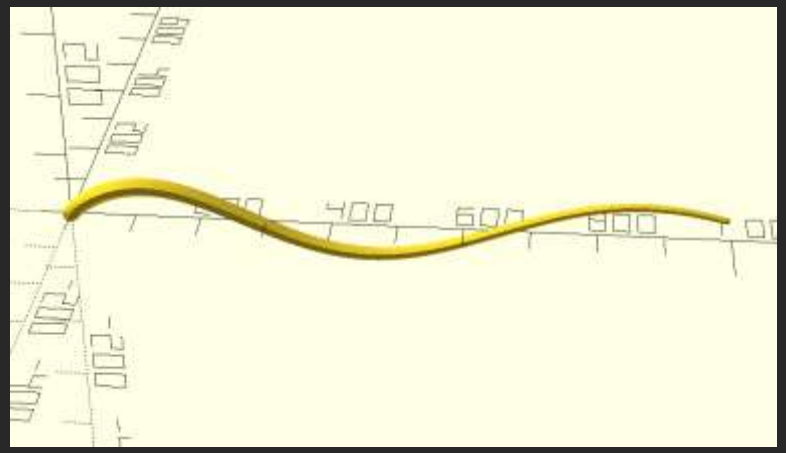
$$f(x) = A\,sin(\varphi x^{d})$$

Control Parameters

- **Amplitude (A):** Determines the height/flow of the curve.
- **Angle Freq (φ):** Controls the tightness of the curve waves.
- **Density (d):** Adjusts the detail level of the shape.

This parametric approach allows for dynamic variation of the beard's length, thickness, and curvature by adjusting a few key values.

Beard Modeling by Eric Guan



Parametric Design Workflow • 3D Modeling Process

Claw Modeling by Eric Guan

Method

Claws were built by creating a base with hulls from spheres, then attaching sharp, tapered cones to form the tips.

Key Functions

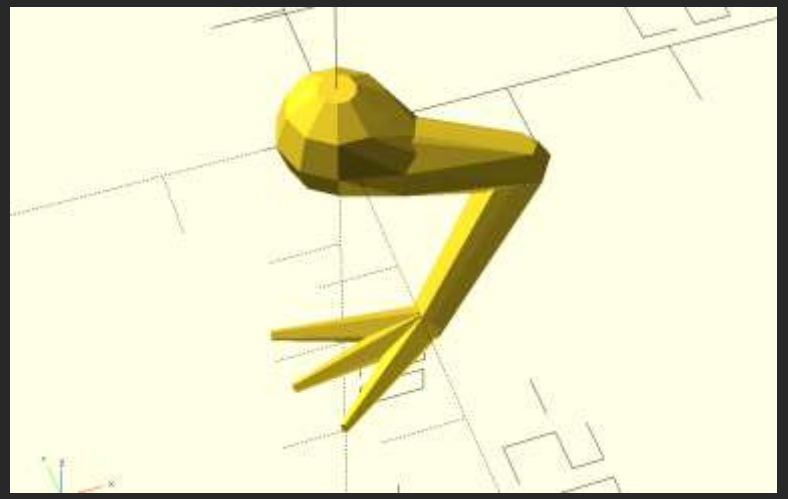
Utilized the **hull()** function to form the solid base structure of the claw before adding the sharp elements.

Parameters

- **size:** Controls overall scale
- **angle:** Defines curvature
- **angle_delta:** Spacing between fingers

Design Insight: By combining organic hull shapes with geometric cones, the model achieves a balance between structural realism and sharp, intimidating aesthetics.

Claw Modeling by Eric Guan



Parametric Design Workflow • 3D Modeling Process

Horn Modeling by Eric Guan

Methodology

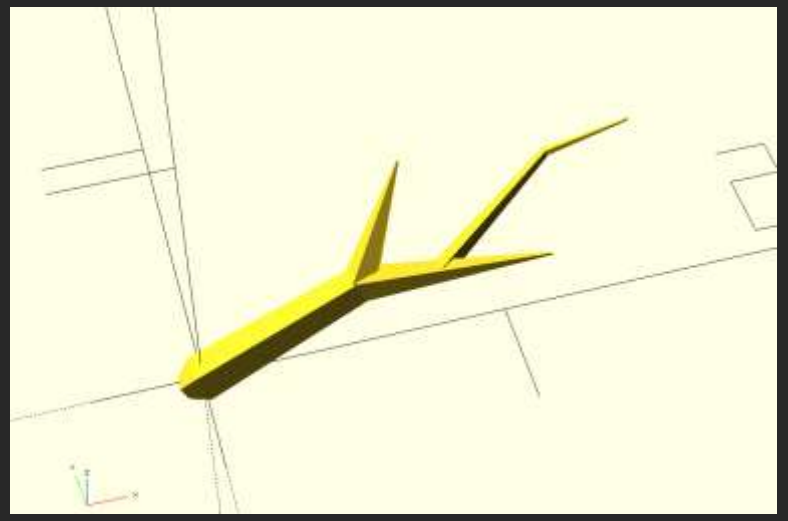
Horns were simple to make. They started with a sphere as the base, and then several smaller, tapered cones were stacked on top to form the long, pointed shape.

Key Parameters

The primary parameter used was **size**, which controlled the overall scale and dimensions of the horn structure.

Process Summary: Sphere (Base) → Stacked Cones (Tapered) → Scaling via Size Control

Horn Modeling by Eric Guan



Parametric Design Workflow • 3D Modeling Process

Head Modeling by Eric Guan

Modeling Method

The head was assembled from pre-made parts (horn, beard) combined with basic shapes (spheres, cones). Boolean operations like **intersection, union, and difference** were used to carve out facial features.

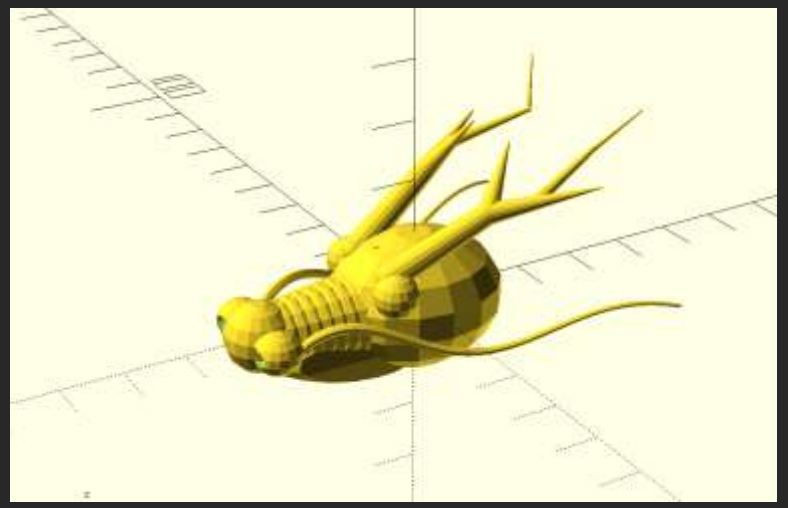
Key Components

Horn, Beard, Spheres, Cones

Control Parameter

The parameter **head_size** was used to control the overall scale of the head model.

Head Modeling by Eric Guan



Parametric Design Workflow • 3D Modeling Process

Tail Modeling by Eric Guan

Methodology

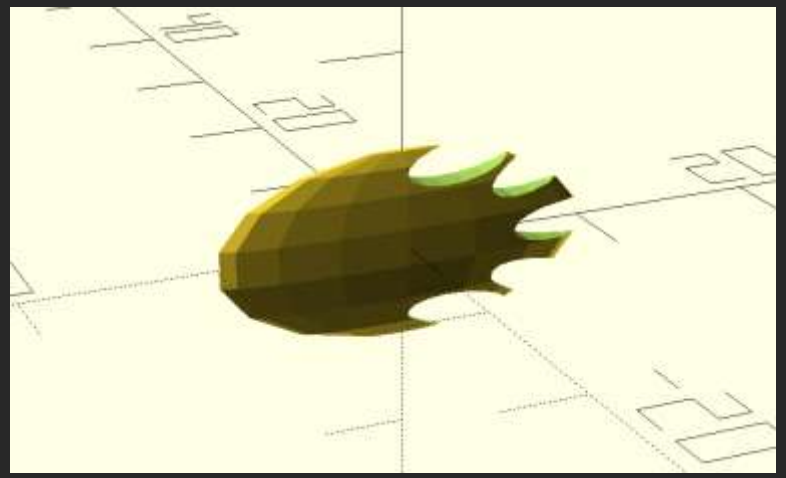
The tail was made by first scaling a sphere to make it long and thin. Then, I used the **difference()** function to subtract several cylinders from it, creating a segmented, tapering look.

Key Functions

- **scale()**: To stretch the sphere into a long shape.
- **difference()**: To carve out segments and create depth.

Design Insight: Combining basic primitives with boolean operations is a powerful technique for creating complex organic forms in 3D modeling.

Tail Modeling by Eric Guan



Parametric Design Workflow • 3D Modeling Process

Body Assembly by Eric Guan

Method: Construction Logic

The main body was formed by creating a long, curved path using a **for-loop** and **hull()** function. Scales were placed along this path, and finally, the head, tail, and claws were attached.

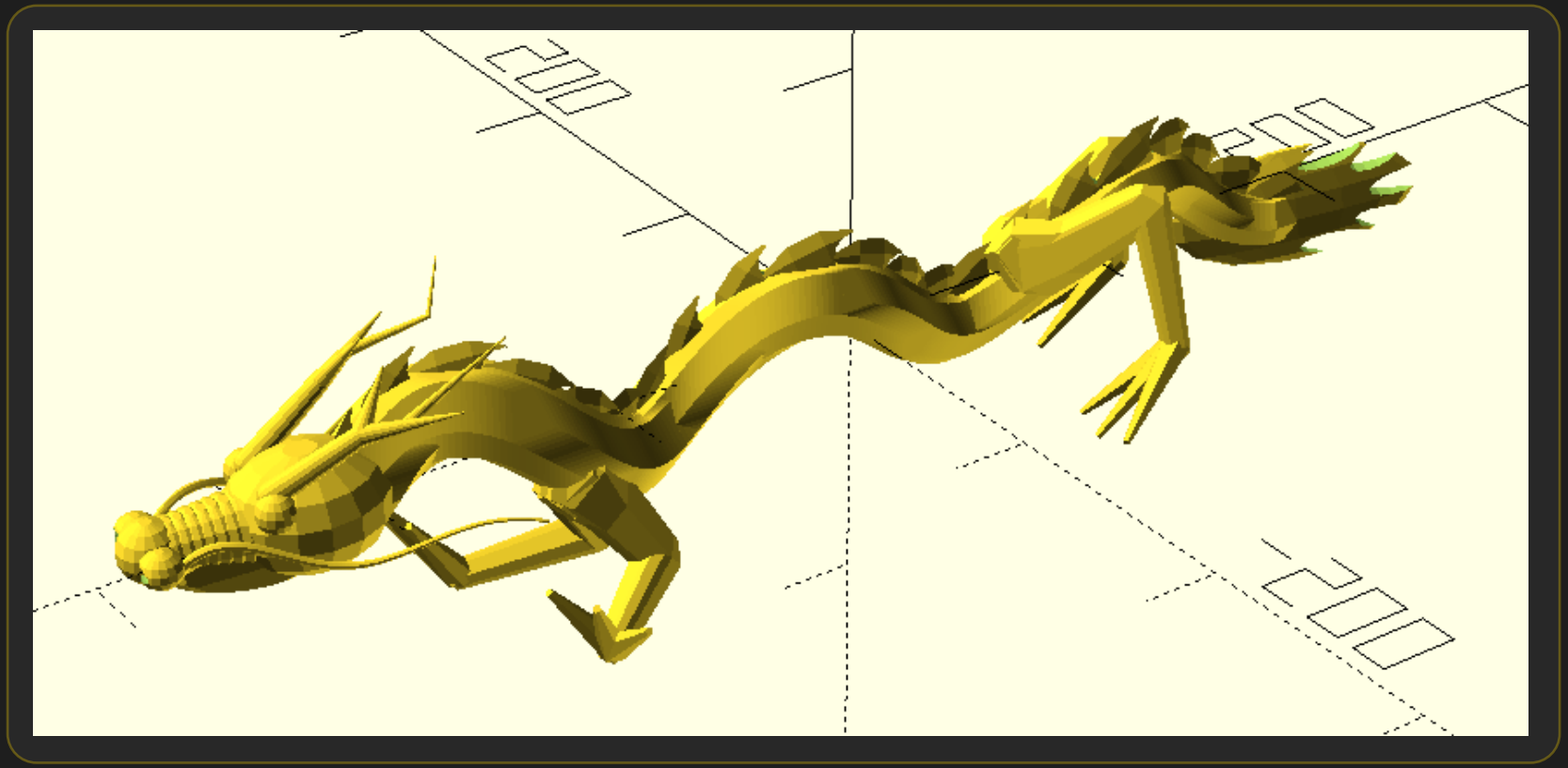
Position & Angle: Natural Flow

The body's curve was controlled by **sine functions** to achieve a natural, flowing pose that mimics organic movement.

Parameters: Key Variables

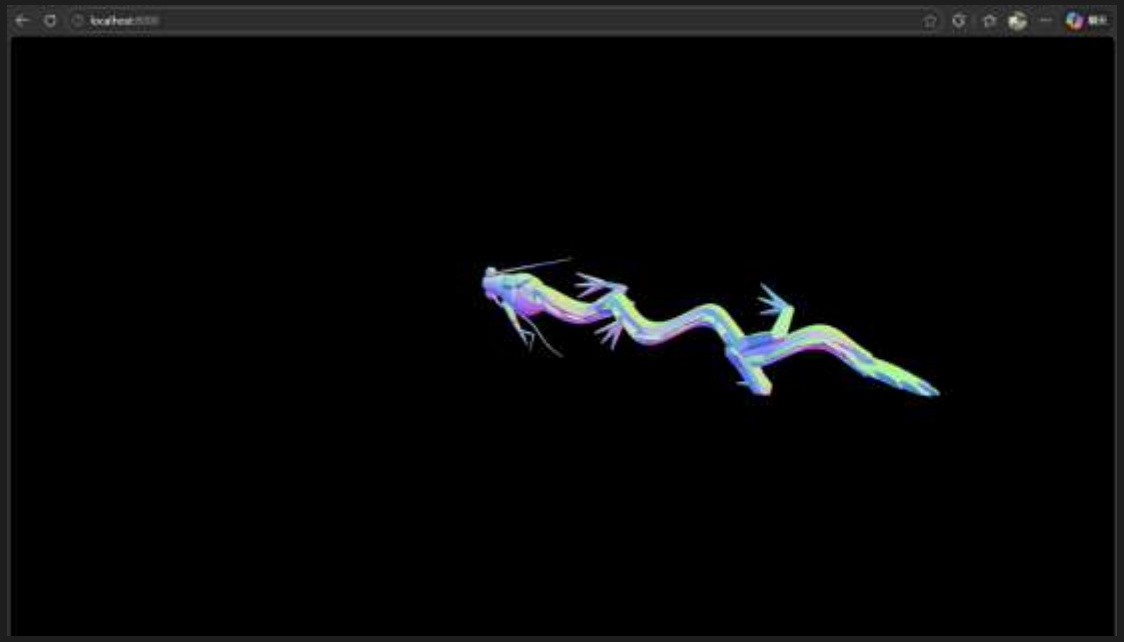
Critical parameters included **radius** (starting size), **amplitude_y/z** (curvature magnitude), and **total_length** (overall body span).

Tail Modeling by Eric Guan

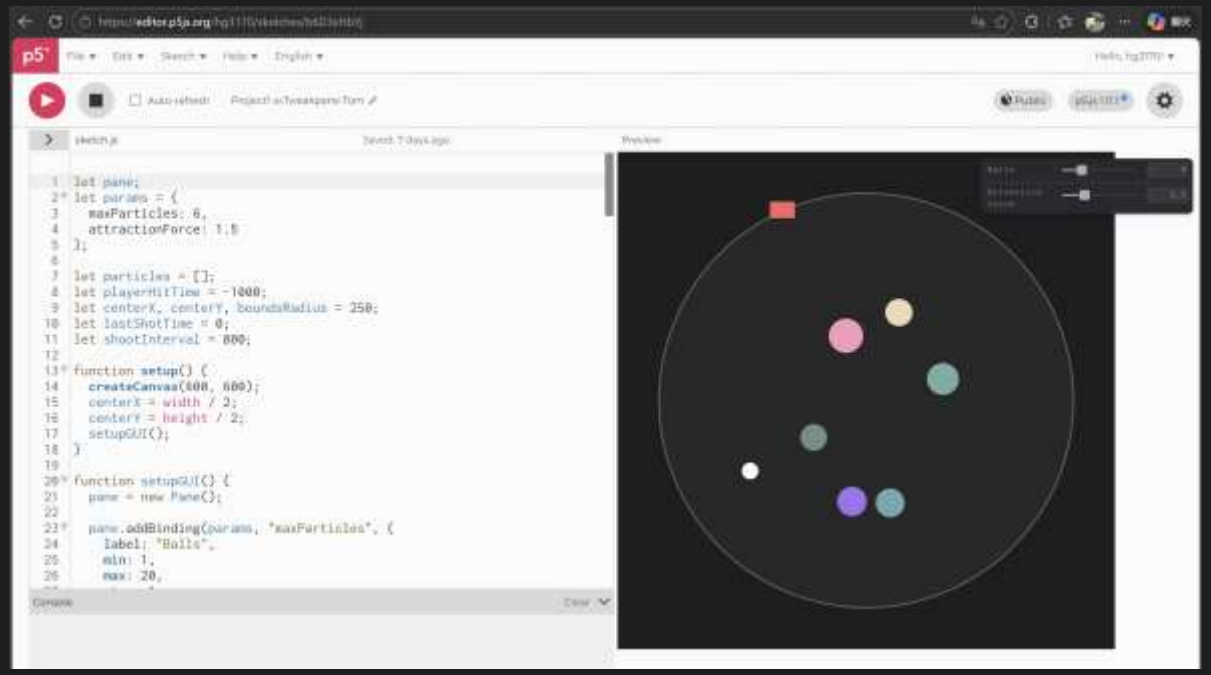


Parametric Design Workflow • 3D Modeling Process

Rendering by Tom Gao



1st Version with default texture



My midterm work on p5.js

Resources I used:

1. My previous experience in rendering/coding
   (p5.js I learned this term is also based on JavaScript)
2. Help & some reading materials from my professor
   (I'll take course using three.js next term)
3. Help from AI

Rendering by Tom Gao

```
<script type="module">
  import * as THREE from 'three';
  import { STLLoader } from 'three/addons/loaders/STLLoader.js';
  import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

  const scene = new THREE.Scene();
  const camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 1000);
  camera.position.z = 5;

  const renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setSize(window.innerWidth, window.innerHeight);
  document.body.appendChild(renderer.domElement);

  const controls = new OrbitControls(camera, renderer.domElement);

  const dragonGroup = new THREE.Group();
  scene.add(dragonGroup);

  const uniforms = {
    uTime: { value: 0.0 }
  };

  const vertexShader = `
    varying vec3 vNormal;
    varying vec3 vViewPosition;
    uniform float uTime;

    void main() {
      vNormal = normalize(normalMatrix * normal);

      vec3 newPosition = position;

      float amplitude = 0.05;
      float frequency = 2.0;
      float speed = 3.0;

      newPosition.x += sin(newPosition.y * frequency + uTime * speed) * amplitude;
      newPosition.z += cos(newPosition.y * frequency + uTime * speed) * amplitude * 0.5;
```

Part of the rendering code

Gemini

3D中国龙项目作业指南

JavaScript

说明：

- `Math.PI / 2` 在数学里就是 90 度。
- `mesh.rotation.x = -Math.PI / 2;` 的意思是：让龙绕着 X 轴往前（或往后）翻滚 90 度。通常这能解决 90% 的 STL 模型躺平问题。

如果加了上面那行，姿势还是不对怎么办？

因为我没看到你队友建的龙到底朝向哪个方向，如果改了 `x` 之后它变成了头朝下或者依然是歪的，你可以试试把那行代码换成围绕 `z` 轴或 `y` 轴旋转：

- **备选方案 A（绕 Z 轴侧翻 90 度）：**
  `mesh.rotation.z = -Math.PI / 2;` （如果反了就去掉负号，改成 `Math.PI / 2`）
- **备选方案 B（绕 Y 轴水平转 90 度）：**
  `mesh.rotation.y = -Math.PI / 2;`

你先加上 `mesh.rotation.x = -Math.PI / 2;` **这行试试！** 保存后刷新网页，看看龙是不是威风凛凛地站起来了？

Help from Gemini

Final Model Showcase

Full View

This is the complete dragon model, showing the body, legs, and tail assembled together.

Head Close-up

This detail shows the horns, eyes, and the beard we created with the hull() function and loops.

Summary & Q&A

Summary

Building the dragon in separate, modular parts made the code easier to manage and modify.

Challenges

The code became a bit disorganized and hard to read in some places.

Future Work

Improve structure for readability and add animations to make the dragon move.

Thank You!